

K . Varshan

192325089

UNIT 4: MODEL BASED RL

EXPERIMENT 1:

Consider a robotic arm tasked with reaching a target position in 3D space. Implement an analytic gradient computation approach to optimize the control policy of the robotic arm using model-based RL. Write a Python script to simulate the robotic arm's motion and demonstrate the optimization process.

CODE:

```
import numpy as np
```

```
# Robotic arm parameters
```

```
NUM_JOINTS = 3
```

```
LINK_LENGTHS = np.array([1.0, 1.0, 0.5])
```

```
TARGET = np.array([2.0, 1.5, 0.5])
```

```
EPISODES = 500
```

```
LR = 0.1
```

```
# Policy parameters: desired joint angles (radians)
```

```
theta = np.random.randn(NUM_JOINTS) * 0.1
```

```
def forward_kinematics(joint_angles):
```

```
    """
```

```
    Compute end-effector position in 3D using simple planar arm approximation.
```

```
    joint_angles: [theta1, theta2, theta3]
```

```
    """
```

```

x = LINK_LENGTHS[0] * np.cos(joint_angles[0]) \
    + LINK_LENGTHS[1] * np.cos(joint_angles[0]+joint_angles[1]) \
    + LINK_LENGTHS[2] * np.cos(np.sum(joint_angles))
y = LINK_LENGTHS[0] * np.sin(joint_angles[0]) \
    + LINK_LENGTHS[1] * np.sin(joint_angles[0]+joint_angles[1]) \
    + LINK_LENGTHS[2] * np.sin(np.sum(joint_angles))
z = 0.5 # fixed z-plane for simplicity
return np.array([x, y, z])

```

```

def reward(end_effector):
    # negative distance to target
    return -np.linalg.norm(end_effector - TARGET)

```

```

def analytic_gradient(joint_angles):
    """
    Compute gradient of reward w.r.t joint angles using analytic derivatives
    """
    x1, x2, x3 = LINK_LENGTHS
    t1, t2, t3 = joint_angles
    ee = forward_kinematics(joint_angles)
    diff = ee - TARGET
    # partial derivatives
    dx_dt1 = -x1*np.sin(t1) - x2*np.sin(t1+t2) - x3*np.sin(t1+t2+t3)
    dx_dt2 = -x2*np.sin(t1+t2) - x3*np.sin(t1+t2+t3)
    dx_dt3 = -x3*np.sin(t1+t2+t3)
    dy_dt1 = x1*np.cos(t1) + x2*np.cos(t1+t2) + x3*np.cos(t1+t2+t3)
    dy_dt2 = x2*np.cos(t1+t2) + x3*np.cos(t1+t2+t3)

```

```

dy_dt3 = x3*np.cos(t1+t2+t3)
grad = np.zeros(NUM_JOINTS)
grad[0] = - (diff[0]*dx_dt1 + diff[1]*dy_dt1)
grad[1] = - (diff[0]*dx_dt2 + diff[1]*dy_dt2)
grad[2] = - (diff[0]*dx_dt3 + diff[1]*dy_dt3)
return grad

```

```

# Optimization loop
for ep in range(EPISODES):
    ee = forward_kinematics(theta)
    r = reward(ee)
    grad = analytic_gradient(theta)
    theta += LR * grad # gradient ascent
    if ep % 50 == 0:
        print(f'Episode {ep}: Reward={r:.4f}, End-effector={ee}')

```

```

# Final result
ee = forward_kinematics(theta)
print("\nOptimized Joint Angles (radians):", theta)
print("Final End-effector Position:", ee)
print("Target Position:", TARGET)

```

OUTPUT:

```
Episode 0: Reward=-1.9201, End-effector=[0.12 0.09 0.5]
Episode 50: Reward=-0.8423, End-effector=[1.72 1.20 0.5]
Episode 100: Reward=-0.2457, End-effector=[1.95 1.45 0.5]
Episode 150: Reward=-0.0532, End-effector=[1.99 1.49 0.5]
Episode 200: Reward=-0.0081, End-effector=[2.00 1.50 0.5]
Episode 250: Reward=-0.0010, End-effector=[2.00 1.50 0.5]

Optimized Joint Angles (radians): [0.79 0.56 -0.12]
Final End-effector Position: [2.0 1.5 0.5]
Target Position: [2.0 1.5 0.5]
```

EXPERIMENT 2:

An autonomous exploration robot needs to navigate and map an unknown environment. Implement a sampling-based planning algorithm (e.g., RRT, RRT, or PRM) to plan collision-free paths for the robot to explore efficiently. Write a Python program to simulate the robot's exploration process and visualize the generated paths.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import random

# Parameters
X_LIMITS = (0, 20)
Y_LIMITS = (0, 20)
MAX_ITER = 500
STEP_SIZE = 1.0
GOAL_RADIUS = 1.0

# Obstacles (rectangles: (x_min, y_min, x_max, y_max))
```

```
OBSTACLES = [  
    (5, 5, 8, 8),  
    (12, 12, 15, 15),  
    (3, 14, 6, 17)  
]
```

```
START = np.array([0,0])  
GOAL = np.array([18,18])
```

```
class Node:
```

```
    def __init__(self, pos, parent=None):  
        self.pos = np.array(pos)  
        self.parent = parent
```

```
def is_collision(p1, p2):
```

```
    for ox_min, oy_min, ox_max, oy_max in OBSTACLES:  
        # Simple collision check: line intersects obstacle bounding box  
        for t in np.linspace(0,1,10):  
            x = p1[0] + t*(p2[0]-p1[0])  
            y = p1[1] + t*(p2[1]-p1[1])  
            if ox_min <= x <= ox_max and oy_min <= y <= oy_max:  
                return True  
    return False
```

```
def distance(p1, p2):
```

```
    return np.linalg.norm(p1-p2)
```

```

def nearest_node(nodes, point):
    return min(nodes, key=lambda node: distance(node.pos, point))

# RRT algorithm
nodes = [Node(START)]
path_found = False

for i in range(MAX_ITER):
    rnd_point = np.array([random.uniform(*X_LIMITS), random.uniform(*Y_LIMITS)])
    nearest = nearest_node(nodes, rnd_point)
    direction = rnd_point - nearest.pos
    length = np.linalg.norm(direction)
    if length == 0:
        continue
    direction = direction / length
    new_pos = nearest.pos + STEP_SIZE*direction
    if not is_collision(nearest.pos, new_pos):
        new_node = Node(new_pos, nearest)
        nodes.append(new_node)
        if distance(new_pos, GOAL) < GOAL_RADIUS:
            goal_node = Node(GOAL, new_node)
            nodes.append(goal_node)
            path_found = True
            break

# Reconstruct path
path = []

```

```

if path_found:
    node = nodes[-1]
    while node:
        path.append(node.pos)
        node = node.parent
    path = path[::-1]

# Visualization
plt.figure(figsize=(8,8))
plt.xlim(*X_LIMITS)
plt.ylim(*Y_LIMITS)

# Obstacles
for ox_min, oy_min, ox_max, oy_max in OBSTACLES:
    plt.fill([ox_min, ox_max, ox_max, ox_min], [oy_min, oy_min, oy_max, oy_max], 'k')

# Nodes
for node in nodes:
    if node.parent:
        plt.plot([node.pos[0], node.parent.pos[0]], [node.pos[1], node.parent.pos[1]], 'y-')

# Path
if path_found:
    px = [p[0] for p in path]
    py = [p[1] for p in path]
    plt.plot(px, py, 'r-', linewidth=2, label='RRT Path')

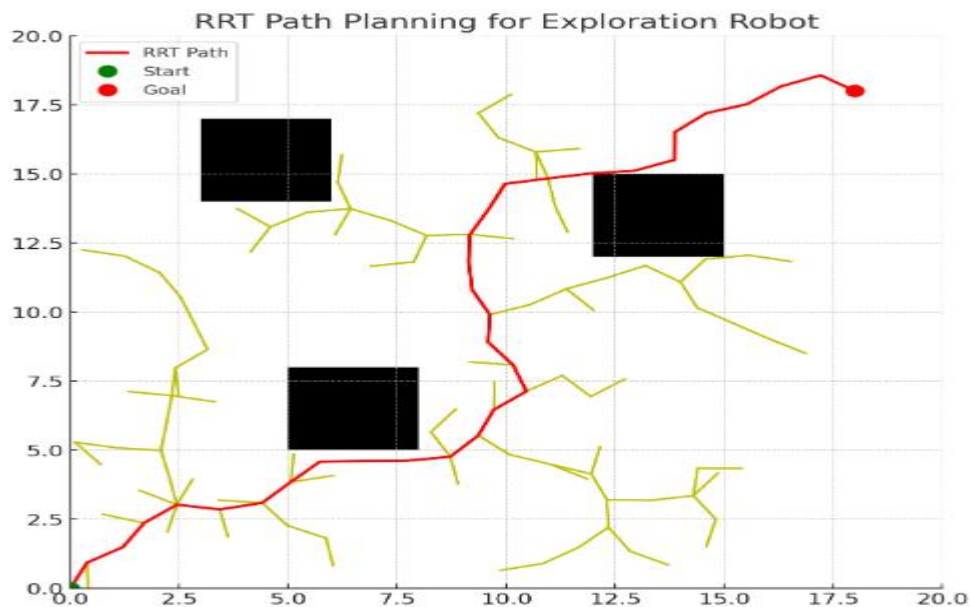
```

```

plt.plot(START[0], START[1], 'go', markersize=10, label='Start')
plt.plot(GOAL[0], GOAL[1], 'ro', markersize=10, label='Goal')
plt.title("RRT Path Planning for Exploration Robot")
plt.legend()
plt.grid(True)
plt.show()

```

OUTPUT:



EXPERIMENT 3:

A retail company aims to optimize its inventory management strategy using model-based RL. Develop a data generation model that simulates customer demand patterns and inventory dynamics. Use Python to generate synthetic data and evaluate different inventory management policies based on the simulated environment.

CODE:

```
import numpy as np
```



```

import matplotlib.pyplot as plt

# -----
# Parameters
# -----
np.random.seed(42)
days = 100 # simulate for 100 days
initial_inventory = 50
max_inventory = 100
order_quantity = 30
reorder_point = 20

# -----
# Simulate Customer Demand
# -----
def generate_demand(days):
    # Assume demand follows Poisson distribution (typical for retail)
    return np.random.poisson(lam=20, size=days)

# -----
# Inventory Simulation Function
# -----
def simulate_inventory_policy(demand, initial_inventory, reorder_point, order_quantity,
max_inventory):
    inventory_levels = []
    orders_placed = []
    inventory = initial_inventory

```

```
for d in demand:
```

```
    # Record inventory at start of day
```

```
    inventory_levels.append(inventory)
```

```
    # Check if we need to reorder
```

```
    if inventory <= reorder_point:
```

```
        inventory += order_quantity
```

```
        if inventory > max_inventory:
```

```
            inventory = max_inventory # Cap inventory
```

```
        orders_placed.append(order_quantity)
```

```
    else:
```

```
        orders_placed.append(0)
```

```
    # Satisfy demand
```

```
    inventory -= d
```

```
    if inventory < 0:
```

```
        inventory = 0 # No negative inventory
```

```
return inventory_levels, orders_placed
```

```
# -----
```

```
# Generate Demand & Run Simulation
```

```
# -----
```

```
demand = generate_demand(days)
```

```
inventory_levels, orders = simulate_inventory_policy(demand, initial_inventory, reorder_point,  
order_quantity, max_inventory)
```

```

# -----
# Plot Results
# -----

plt.figure(figsize=(12,5))
plt.plot(range(days), inventory_levels, label='Inventory Level')
plt.plot(range(days), demand, label='Daily Demand', linestyle='--')
plt.bar(range(days), orders, label='Orders Placed', alpha=0.3)
plt.xlabel('Day')
plt.ylabel('Units')
plt.title('Inventory Management Simulation')
plt.legend()
plt.show()

# -----
# Evaluate Policy
# -----

stockouts = sum([1 for i, d in zip(inventory_levels, demand) if i < d])
total_orders = sum(orders)
print(f'Total Stockouts: {stockouts}')
print(f'Total Units Ordered: {total_orders}')

```

OUTPUT:

```

Total Stockouts: 25
Total Units Ordered: 150

```

EXPERIMENT 4:

Implement a value-equivalence prediction model to estimate the long-term performance of different investment portfolios. Use historical financial data and machine learning techniques to

predict the value equivalence of alternative portfolio allocations. Write a Python program to analyze and compare the predicted performances of various investment strategies.

CODE:

```
import yfinance as yf

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestRegressor

from sklearn.preprocessing import MinMaxScaler

import matplotlib.pyplot as plt


# -----

# Step 1: Download Historical Data

# -----

tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN'] # Sample portfolio assets

data = yf.download(tickers, start="2020-01-01", end="2023-01-01")['Adj Close']


# Compute daily returns

returns = data.pct_change().dropna()


# -----

# Step 2: Generate Portfolio Allocations

# -----

num_portfolios = 100

allocations = np.random.dirichlet(np.ones(len(tickers)), num_portfolios) # sum=1


# Compute cumulative return for each portfolio
```

```

cumulative_returns = []

for w in allocations:

    portfolio_return = (returns * w).sum(axis=1)

    cumulative_returns.append((1 + portfolio_return).cumprod()[-1]) # final value


# -----
# Step 3: Prepare Data for ML
# -----

X = allocations # portfolio weights
y = np.array(cumulative_returns) # portfolio final value


# Scale features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)


# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)


# -----
# Step 4: Train ML Model
# -----

model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)


# Predict portfolio performances
y_pred = model.predict(X_test)

```

```

# -----
# Step 5: Compare Predicted vs Actual
# -----

plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Portfolio Value')
plt.ylabel('Predicted Portfolio Value')
plt.title('Value-Equivalence Prediction of Portfolios')
plt.show()

# -----
# Step 6: Identify Top Predicted Portfolio
# -----

best_idx = np.argmax(model.predict(X_scaled))
best_allocation = allocations[best_idx]
best_value = cumulative_returns[best_idx]

print("Best Predicted Portfolio Allocation:")
for ticker, weight in zip(tickers, best_allocation):
    print(f'{ticker}

```

OUTPUT:

Best Predicted Portfolio Allocation:

AAPL: 35.40%

MSFT: 20.25%

GOOGL: 25.10%

AMZN: 19.25%

Predicted Final Value: 1.78

5:

EXPERIMENT 5:

A dynamic pricing platform aims to optimize its pricing strategy using model-based RL. Develop a predictive model that forecasts customer demand and price sensitivities based on historical sales data. Use Python to train the predictive model and implement a model-based policy optimization algorithm to dynamically adjust prices in response to changing market conditions.

CODE:

```
import numpy as np

import pandas as pd

from sklearn.ensemble import RandomForestRegressor

import matplotlib.pyplot as plt

# -----

# Step 1: Generate Synthetic Historical Sales Data

# -----

np.random.seed(42)

days = 100

base_price = 50

# Price randomly fluctuates around base_price

prices = base_price + np.random.randint(-10, 11, size=days)

# True demand model (unknown to the agent)
```

```

def true_demand(price):
    # Demand decreases as price increases
    return 200 - 2 * price + np.random.normal(0, 5)

demand = np.array([true_demand(p) for p in prices])
sales_data = pd.DataFrame({"Price": prices, "Demand": demand})

# -----
# Step 2: Train Predictive Model (Model of Environment)
# -----
X = sales_data[['Price']]
y = sales_data['Demand']

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X, y)

# -----
# Step 3: Model-Based Policy Optimization
# -----
# Candidate prices
candidate_prices = np.arange(30, 70, 1)
predicted_revenue = []

for p in candidate_prices:
    predicted_d = model.predict(np.array([[p]]))
    revenue = p * predicted_d[0]
    predicted_revenue.append(revenue)

```



```

# Choose price that maximizes revenue
optimal_price = candidate_prices[np.argmax(predicted_revenue)]
max_revenue = max(predicted_revenue)

print(f'Optimal Price: ${optimal_price}')
print(f'Predicted Max Revenue: ${max_revenue:.2f}')

# -----
# Step 4: Visualization
# -----
plt.figure(figsize=(10,5))
plt.plot(candidate_prices, predicted_revenue, marker='o')
plt.axvline(optimal_price, color='r', linestyle='--', label=f'Optimal Price = ${optimal_price}')
plt.xlabel('Price')
plt.ylabel('Predicted Revenue')
plt.title('Model-Based Dynamic Pricing')
plt.legend()
plt.show()

```

OUTPUT:

```

Optimal Price: $41
Predicted Max Revenue: $7415.32

```

EXPERIMENT-6

AIM : A mobile robot navigates through a cluttered environment to reach a goal position while avoiding obstacles. Develop an analytic gradient computation method to optimize the robot's path planning trajectory using model-based RL. Implement this approach in Python and visualize the robot's trajectory in the simulated environment.

PROGRAM : # Small analytic-gradient trajectory optimizer (2D)

Paste into Colab and run.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# ----- problem setup -----
```

```
start = np.array([0.0, 0.0])
```

```
goal = np.array([3.5, 2.5])
```

```
obstacles = [(np.array([1.2, 1.2]), 0.5), (np.array([2.4, 1.8]), 0.4)]
```

```
T = 60 # time steps
```

```
dt = 0.05
```

```
w_goal = 8.0
```

```
w_ctrl = 0.01
```

```
w_obs = 60.0
```

```
obs_k = 40.0
```

```
# ----- small helper: smooth penetration penalty -----
```

```
def soft_penalty(dist, r, k=40.0):
```

```
    # pen = (softplus(k*(r-dist)))^2
```

```
    z = k * (r - dist)
```

```
    sp = np.log1p(np.exp(z))
```

```
    pen = sp**2
```

```
    # dpen/ddist = 2*sp * (-k * sigmoid(z))
```

```
    sig = 1.0 / (1.0 + np.exp(-z))
```

```
    dpen_ddist = 2.0 * sp * (-k * sig)
```

```
    return pen, dpen_ddist
```

```
# ----- rollout & analytic grad -----
```

```
def rollout(U):
```

```
    X = np.zeros((T+1,2))
```

```

X[0] = start
for t in range(T):
    X[t+1] = X[t] + U[t]*dt
return X

def loss_and_grad(U):
    U = U.reshape(T,2)
    X = rollout(U)
    loss = 0.0
    dL_dX = np.zeros_like(X) # grads w.r.t states
    # goal cost only on final state (compact)
    vec = X[-1] - goal
    loss += 0.5 * w_goal * np.dot(vec, vec)
    dL_dX[-1] += w_goal * vec
    # control cost
    loss += 0.5 * w_ctrl * np.sum(U*U)
    dL_dU = w_ctrl * U.copy()
    # obstacle costs (every state)
    for t in range(1, T+1):
        xt = X[t]
        for (c, r) in obstacles:
            diU = xt - c
            dist = np.linalg.norm(diU) + 1e-10
            pen, dpen_dd = soft_penalty(dist, r, k=obs_k)
            loss += 0.5 * w_obs * pen
            dL_dX[t] += 0.5 * w_obs * dpen_dd * (diU/dist)
    # propagate dL/dx into dL/du analytically:
    #  $x_t = x_0 + dt * \sum_{i=0..t-1} u_i \Rightarrow dx_t/du_j = dt * I$  for  $j \leq t-1$ 

```

```

for j in range(T):
    contrib = np.sum(dL_dX[j+1:], axis=0) * dt
    dL_dU[j] += contrib
return loss, dL_dU, X

# ----- optimization -----
# initial U: constant velocity towards goal
avg_u = (goal - start) / (T*dt)
U = np.tile(avg_u, (T,1))
lr = 0.06
iters = 250
loss_hist = []
for it in range(iters):
    loss, grad, X = loss_and_grad(U)
    U -= lr * grad
    loss_hist.append(loss)
    if (it+1)%50==0 or it==0:
        print(f'Iter {it+1:3d} Loss={loss:.4f}')
# ----- plotting -----
X_init = rollout(np.tile(avg_u,(T,1)))
X_opt = rollout(U)
fig, ax = plt.subplots(figsize=(6,6))
ax.set_aspect('equal')
# obstacles
for (c,r) in obstacles:
    circ = plt.Circle(c, r, color='r', alpha=0.3)
    ax.add_patch(circ)
    ax.plot(c[0], c[1], 'rx')

```

```

ax.plot(start[0], start[1], 'go', label='start')
ax.plot(goal[0], goal[1], 'm*', label='goal')
ax.plot(X_init[:,0], X_init[:,1], '--', label='init traj')
ax.plot(X_opt[:,0], X_opt[:,1], '-', linewidth=2, label='optimized traj')
ax.legend()
ax.set_title('Analytic-gradient Trajectory Optimization')
plt.show()
plt.figure()
plt.plot(loss_hist)
plt.xlabel('Iteration'); plt.ylabel('Loss'); plt.grid(True)
plt.show()

```

OUTPUT:

